

ITCS381 Intro. to Multimedia System P5.js
Project of Manaw thang suan yang mai preaw
thao puak phom a kub

Phuriwat Angkoondittaphong Rujiphat Charatvaraphan
6388003 6388012

Bunradar Chatchaiyadech
6388185

March 27, 2022



Figure 1: Final version of our animation

1 Introduction

Our project based on the idea of many comets like object orbit around their center. So, we start to think about how to draw circle in the canvas and recognise that the idea of polar coordinate system can be use for this project.

2 Circle on Polar coordinate system and conversion to XY coordinate system

The polar coordinate system is two-dimensional coordinate system that use length and angle to plot. Function in polar coordinate system can be express as function of distance from the origin over angle,

$$r(\theta)$$

Where r is distance from the origin, θ is angle.

Could try the example in desmos [here](#). Our idea is to use time as an angle since time continuously increases. We can adjust the rotation speed by adding some multiplicative factors later on.

However, the only polar coordinate system is not enough. We also have to convert it into a parametric function on XY coordinate system as well since P5.js use it. Luckily conversion can be done easily using the following formulae.

$$x(\theta) = r(\theta)\cos(\theta)$$

$$y(\theta) = r(\theta)\sin(\theta)$$

The equation for circle on polar coordinate system is

$$r(\theta) = n$$

Where $n \in \mathbf{R}$. $|n|$ represents radius of the circle.

(can see that θ does not affect the distance from the origin. Hence, we got the circle.)

Convert to XY coordinate system. Substitute $r \rightarrow x, y$ we get

$$x(\theta) = n\cos(\theta), y(\theta) = n\sin(\theta)$$

Where n is radius of the circle.

We can generalize further by substituting the term $\theta = 2\pi ft + \theta_0$. We get

$$x(t) = n\cos(c \cdot 2\pi ft + \theta_0), y(t) = n\sin(c \cdot 2\pi ft + \theta_0)$$

where $n \in \mathbf{R}^+$ is radius of the circle, $f \in \mathbf{R}^+$ is frequency, $\theta_0 \in \mathbf{R}$ is initial angle, $c \in \{-1, 1\}$ is clockwise or anti-clockwise, and $t \in \mathbf{R}^+$ is time .

3 Implementation of Comet

3.1 Version 1

We can write a JavaScript function that generates a parametric function easily. Here's Version 1

```

function generate_circle_eq_v1(init, frequency, orbit_radius, cc){
  return function(t){
    let theta = 2 * PI * frequency * cc * t + init;
    return [orbit_radius * cos(theta), orbit_radius * sin(theta)];
  }
}

```

Where init is θ_0 , frequency is f , orbit_radius is n , and cc is c . With version 1, we got position of XY position that orbit on the circle. Here's [demo](#).

3.2 Version 2

Version 1 isn't enough. We want comet like object not only one circle. So we try to return many positions to get the tail of the comet and return radius of the drawing circle as well. Which we use the exponential (decay) formula to scale to the shape of the converging tail.

$$r_i = r_0 s^i$$

Where r_i is radius of drawing circle at index i , r_0 is head comet's radius, $s \in (0, 1]$ is scaling factor for radius, and i running number for each tail circle. Here's Version 2

```

const dt = 0.05
const radius_scale = 0.98
function generate_circle_eq_v2(init, frequency, orbit_radius, no_circle, cc, radius){
  return function(t){
    let theta;
    let ar = [];
    for(let i = 0; i < no_circle; i++){
      theta = 2 * PI * frequency * cc * (t - dt * i) + init;
      ar.push([orbit_radius * cos(theta),
               orbit_radius * sin(theta),
               radius*pow(radius_scale, i)]);
    }
    return ar;
  }
}

```

Where init is θ_0 , frequency is f , orbit_radius is n , cc is c , no_circle is number of drawing circles, and radius is the head's radius and use for determine tail's radius. With version 2 we get the tail of the comet. Here's version 2 [demo](#).

3.3 Version 3

In version 3 we try further to encapsulate everything into one function.

```

const dt = 0.05
const radius_scale = 0.98
function generate_circle_eq_v3(init,
                                frequency,
                                orbit_radius,
                                no_circle,
                                cc,
                                radius,
                                color){
  return function(t){
    let theta;
    let ar = [];
    for(let i = 0; i < no_circle; i++){
      theta = 2 * PI * frequency * cc * (t - dt * i) + init;
      ar.push([orbit_radius * cos(theta),
               orbit_radius * sin(theta),
               radius*pow(radius_scale, i),
               color]);
    }
    return ar;
  }
}

```

Where init is θ_0 , frequency is f , orbit_radius is n , cc is c , no_circle is number of drawing circles, radius is the head's radius and use for determine tail's radius, and color is color of comet. With version 3 we also have color of the comet encapsulate within one function call. Here's version 3 [demo](#). Hence, we use version 3 for the project.

4 Randomness make it cooler

We have more ideas that implement repetition concept on `draw()` function to random the initial state θ_0 of each circle starting point by using `(Math.random()*2 $\pi$ )` on attribute `init`. And repeat the different starting point based on the time that we adjusted by randoming `init` attribute on `draw()` function. You can play with our demo [here](#).

5 Final version on P5.js (TLDR)

We can duplicate these initial two circles direction in many ways based on the plan that we will create the animation. So, from the benefits of duplication, we decorate more beautiful circles by adjusting each circle's attributes. Here's overall code

```

const dt = 0.050;
const radius_scale = 0.98;

```

```

let OT;
let function_list;
let params;

function generate_circle_eq(init, frequency, orbit_radius, no_circle, cc, radius, color){
  return function(t){
    let theta;
    let ar = [];
    for(let i = 0; i < no_circle; i++){
      theta = 2 * PI * frequency * cc * (t - dt * i) + init;
      ar.push([orbit_radius * cos(theta),
              orbit_radius * sin(theta),
              radius*pow(radius_scale, i),
              color]);
    }
    return ar;
  }
}

function setup() {
  createCanvas(400, 400);
  noStroke();
  OT = Math.floor(millis() / 1000);
}

function draw() {
  background(255);
  let t = millis() / 1000;

  if(Math.floor(t) >= OT){
    params = [
      [(Math.random()*2*PI), 0.4, 5, 20, 1, 5, [0, 0, 0]],
      [(Math.random()*2*PI), 0.041, 15, 110, -1, 6, [250, 0, 250]],
      [(Math.random()*2*PI), 0.042, 30, 120, 1, 7, [220, 0, 220]],
      [(Math.random()*2*PI), 0.043, 45, 130, -1, 8, [190, 0, 190]],
      [(Math.random()*2*PI), 0.044, 60, 140, 1, 9, [160, 0, 160]],
      [(Math.random()*2*PI), 0.045, 80, 150, -1, 9, [130, 0, 130]],
      [(Math.random()*2*PI), 0.046, 100, 160, 1, 9, [110, 0, 110]],
      [(Math.random()*2*PI), 0.047, 120, 170, -1, 10, [80, 0, 80]],
      [(Math.random()*2*PI), 0.048, 140, 180, 1, 10, [50, 0, 50]],
      [(Math.random()*2*PI), 0.048, 160, 190, -1, 10, [10, 0, 10]]
    ];

    function_list = params.map(x => generate_circle_eq(...x));
    OT = Math.floor(t+0.8);
  }
}

```

```

let center_circles = function_list.map(f => f(t));
for(let a = 0; a < center_circles.length; a++){
  for(let i = 0; i < center_circles[a].length; i++){
    let [x,y,r,c] = center_circles[a][i];
    fill(color(...c));
    circle(x + width/2, y + height/2, r);
  }
}
}

```

In the setup function, we created a canvas element, and set the dimensions of it in pixels (400×400 pixels). Then, we set up the background color and disabled drawing the stroke (outline). And the last line, OT is equal to the rounded down number of the number of milliseconds divided by a thousand.

In the draw function, we draw a white background and set t equal to the number of milliseconds divided by a thousand. Then, use the if statement to generate the circles. In the if statement, there is a function_list, which uses a params array to generate circles by using generate_circle_eq function, which needs seven parameters to create each circle. There are initial, frequency, orbital radius, number of circles, cc which 1 means clockwise and -1 means anticlockwise, radius of the circle, and color. And there is a OT to define how long each pattern runs for more interesting animation. And then show them and set them at the center of the canvas.

You can play with it [here](#).